Arena RPG con POO Avanzada, Colecciones y Genéricos

Taller Práctico Integrador

1. Sección 1: Arquitectura y Diagrama UML (Unidad 2.3)

1.1 Especificación del Diagrama UML Simplificado

Para modelar la arquitectura de nuestra arena RPG, utilizaremos una jerarquía orientada a objetos que combine abstracción pura y herencia de estado. La estructura visual de componentes y contratos del sistema se representa de la siguiente manera:

- `<<interface>>` `**ICombatiente**` *(Contrato comportamental)*
  - ▲
  - | *(implementa)*
  - |
- `<<abstract>>` `**Personaje**` *(Clase base abstracta)*
  - ├── `**Guerrero**` *(Subclase concreta - Especialización de combate físico)*
  - └── `**Mago**` *(Subclase concreta - Especialización de combate mágico)*

1.2 Diseño de la Clase Abstracta `Personaje` e Interfaz `ICombatiente`

La interfaz `**ICombatiente**` establece las acciones defensivas mínimas que cualquier entidad interactiva debe ofrecer en el área de combate. Por otro lado, la clase abstracta `**Personaje**` consolida el estado común (atributos) y la lógica compartida, forzando a las subclases a definir la mecánica específica de ataque especial.

- **Interfaz** `**ICombatiente**` **:**
  - `**void defender(int danoRecibido)**` : Método que procesa la reducción de daño según la armadura o mitigación base de la entidad.
- **Clase Abstracta** `**Personaje**` **(Implementa** `**ICombatiente**` **):**
  - **Atributos Protegidos (** `**protected**` **):**
    - `**protected String nombre**` : Identificador único del combatiente.
    - `**protected int vida**` : Puntos de salud actuales.
    - `**protected int ataqueBase**` : Valor numérico base para calcular el daño físico o mágico.
    - `**protected int posicionX**` : Coordenada horizontal en el tablero de juego.
    - `**protected int posicionY**` : Coordenada vertical en el tablero de juego.
  - **Método Abstracto:**
    - `**public abstract void realizarAtaqueEspecial(Personaje objetivo)**` : Método sin implementación en la clase base; obliga a cada clase derivada a definir su propia lógica de ataque único.

- **Métodos Concretos Compartidos:**
  - `**public boolean estaVivo()**`: Retorna `true` si `vida > 0`.
  - `**public void recibirDano(int cantidad)**`: Descuenta la salud del personaje garantizando que no descienda de cero.
  - `**public String getNombre()**` y `**public int getVida()**`: Métodos accesores estándar (*getters*).

¿Por qué usar una Clase Abstracta frente a una Interfaz en este contexto? Empleamos la interfaz `ICombatiente` para definir un *contrato de comportamiento puro* (`defender()`) desligado de cualquier implementación. Sin embargo, recurrimos a la clase abstracta `Personaje` porque nuestros personajes comparten *estado común* (atributos como `nombre`, `vida`, `posicionX`, `posicionY`) y *comportamiento concreto reutilizable* (como `estaVivo()` y `recibirDano()`). Las interfaces en Java no pueden mantener estado de instancia (no poseen atributos no estáticos), por lo que la clase abstracta es la herramienta adecuada para evitar la duplicación de código de estado y métodos base.

1.3 Especialización de Subclases: `Guerrero` y `Mago`

Concepto de POO	Aplicación en Guerrero/Mago	Beneficio Técnico para el Alumno
Herencia (`**extends**`)	Declaración explícita de `public class Guerrero extends Personaje` y `public class Mago extends Personaje`.	Permite reutilizar todos los atributos base (`nombre`, `vida`) sin volver a declararlos en las subclases.
Constructor Padre (`**super()**`)	Invocación en la primera línea del constructor hijo: `super(nombre, vida, ataqueBase, posX, posY);`.	Garantiza la correcta inicialización del estado heredado antes de asignar atributos específicos de la subclase.
Atributos Específicos	`Guerrero` añade `protected int fuerzaEscudo`; `Mago` añade `protected int puntosMana`.	Modela la especialización del dominio extendiendo el estado básico con características únicas.
Sobreescritura (`**@Override**`)	Redefinición del método `realizarAtaqueEspecial(Personaje objetivo)` adaptado al daño de área o uso de maná.	Aplica polimorfismo dinámico: la máquina virtual de Java (JVM) resuelve en tiempo de ejecución qué método ejecutar.
Sobrecarga de Métodos	Creación de `atacar(Personaje p)` y `atacar(Personaje p, boolean modoFuria)` dentro de la misma clase.	Proporciona flexibilidad en la interfaz pública de la clase mediante polimorfismo estático en tiempo de compilación.

Una vez definida la jerarquía rígida de clases e interfaces, debemos resolver cómo almacenar y gestionar estas entidades dinámicamente en memoria durante la batalla.

2. Sección 2: Gestión de Escuadrón con `ArrayList<Personaje>` (Unidad 2.4)

2.1 Refactorización: De Variables Aisladas a Colecciones Dinámicas

El manejo de combatientes mediante variables individuales (`Personaje p1` , `Personaje p2` , `Personaje e1`) genera código rígido, propenso a errores y no escalable. La adopción de la colección `ArrayList<Personaje>` permite reestructurar el escuadrón en un contenedor dinámico.

Criterio	Antes (Variables Aisladas)	Después (`ArrayList<Personaje>`)
Escalabilidad	Fija. Para agregar un quinto personaje se requiere modificar la firma de métodos y crear variables manuales.	Dinámica. El escuadrón puede crecer o decrecer en tiempo de ejecución sin alterar el diseño.
Procesamiento de Turnos	Repetitivo. Requiere invocaciones explícitas e individuales: `p1.atacar()` , `p2.atacar()` , `p3.atacar()` .	Iterativo. Se procesan N personajes mediante un único bucle polimórfico limpio.
Baja de Unidades	Compleja. Requiere asignar `null` a variables y validar condicionales aislados de supervivencia.	Eficiente. Eliminación directa del elemento de la lista mediante invocaciones a la API de colecciones.

2.2 Operaciones de Escuadrón y Recorrido Polimórfico

Siga estos pasos para implementar la gestión dinámica de su escuadrón:

1. **Declaración e Inicialización:** Instancie la lista dinámica utilizando la referencia abstracta `List` :

```
List<Personaje> escuadron = new ArrayList<>();
```

1. **Adición de Elementos Polimórficos (`**add**`):** Agregue subclases concretas a la colección base de tipo `Personaje` :

```
escuadron.add(new Guerrero("Thorin", 120, 15, 0, 0, 10));
escuadron.add(new Mago("Gandalf", 80, 25, 0, 1, 50));
```

1. **Consulta e Inspección (`**get**` , `**size**`):** Obtenga la cantidad de unidades activas mediante `escuadron.size()` y acceda a posiciones específicas con `escuadron.get(indice)` .
2. **Eliminación de Caídos (`**removeIf**` / `**remove**`):** Remueva del escuadrón las unidades cuya vida sea menor o igual a cero:

```
escuadron.removeIf(p -> !p.estaVivo());
```

1. **Recorrido Polimórfico mediante `**for-each**`:** Ejecute las acciones de combate sin importar la subclase concreta subyacente:

```
for (Personaje combatiente : escuadron) {
    // Invocación polimórfica: ejecuta el ataque especial según la instancia real (Guerrero o Mago)
    combatiente.realizarAtaqueEspecial(enemigoObjetivo);
}
```

Con las entidades agrupadas dinámicamente en listas, es momento de diseñar el almacenamiento seguro de sus ítems de soporte y la ubicación espacial dentro del terreno de combate.

3. Sección 3: Contenedor Genérico `Inventario<T>` y Tablero 2D (Unidad 2.4)

3.1 Implementación del Contenedor Genérico `Inventario<T>`

Para evitar crear múltiples clases de inventarios (`InventarioPociones`, `InventarioEquipamiento`), diseñaremos una clase genérica parametrized sobre un tipo `<T>`.

Seguridad de Tipos (*Type Safety*): Al definir `public class Inventario<T>`, el compilador de Java garantiza que solo se puedan almacenar objetos del tipo especificado en la parametrización (por ejemplo, `Inventario<Pocion>`). Esto previene la contaminación de datos en tiempo de compilación.

Eliminación del Casteo Explicito: En colecciones no genéricas obsoletas (basadas en `Object`), recuperar un elemento requería un casteo explícito propenso a errores: `Pocion p = (Pocion) inventario.obtener(0);`. Con genéricos, Java realiza la comprobación en compilación y asigna el tipo automáticamente: `Pocion p = inventario.obtener(0);`.

Estructura Interna del Genérico: La clase genérica administra internamente una lista de tipo `T`:

```
public class Inventario<T> {
    private List<T> items = new ArrayList<>();
    public void agregar(T item) { items.add(item); }
    public T obtener(int index) { return items.get(index); }
    public List<T> getItems() { return items; }
}
```

3.2 Estructura y Representación del Tablero de Batalla (Matriz 2D)

El campo de juego se modela mediante una matriz bidimensional de caracteres (`char[][] tablero = new char[5][5];`), donde cada celda representa un cuadrante del mapa.

Representación del Tablero en Consola (Matriz 5x5):

	Columna 0	Columna 1	Columna 2	Columna 3	Columna 4
Fila 0	`[ G ]`	`[ . ]`	`[ . ]`	`[ . ]`	`[ E ]`
Fila 1	`[ . ]`	`[ M ]`	`[ . ]`	`[ . ]`	`[ . ]`
Fila 2	`[ . ]`	`[ . ]`	`[ C ]`	`[ . ]`	`[ . ]`
Fila 3	`[ . ]`	`[ . ]`	`[ . ]`	`[ . ]`	`[ . ]`
Fila 4	`[ E ]`	`[ . ]`	`[ . ]`	`[ . ]`	`[ E ]`

Leyenda: `_[G]_` *Guerrero,* `_[M]_` *Mago,* `_[E]_` *Enemigo,* `_[C]_` *Cofre/Tesoro,* `_[.]_` *Celda Vacía.*

Algoritmo de Renderizado con Bucles Anidados:

```
public void mostrarTablero(char[][] matriz) {
    for (int i = 0; i < matriz.length; i++) {
        for (int j = 0; j < matriz[i].length; j++) {
            System.out.print("[" + matriz[i][j] + "]");
        }
        System.out.println(); // Salto de línea por cada fila
    }
}
```

Una vez articulados la jerarquía de personajes, la gestión de escuadrones, el contenedor genérico de ítems y la matriz del mapa, integraremos todos estos componentes dentro de la lógica del bucle principal de combate.

4. Sección 4: Reto Integrador de Batalla en Consola

4.1 Flujo de Control y Simulación del Combate

El estudiante debe estructurar la simulación del combate combinando un bucle de control general y menús de decisión por turnos:

1. **Inicialización:** Instanciar el mapa 2D, el `ArrayList<Personaje>` para los héroes, la lista de enemigos y el `Inventario<Pocion>` .
2. **Ciclo Principal de Batalla (`**do-while**`):**
   - Evaluar el estado del juego: ejecutar mientras el escuadrón del jugador tenga unidades vivas **y** queden enemigos activos.

3. **Fase de Renderizado y Estado:** Imprimir en consola el estado del tablero 2D y las estadísticas puntuales de vida y maná de cada personaje.
4. **Turno del Jugador - Menú de Opciones (`**switch**`):**
   - Iterar sobre cada héroe vivo utilizando un bucle `for-each` .
   - Desplegar menú interactivo por consola mediante `Scanner` :
     - `Opción 1: Atacar` -> Llama a `realizarAtaqueEspecial()` sobre un enemigo seleccionado.
     - `Opción 2: Usar Inventario` -> Accede al `Inventario<Pocion>` para recuperar vida.
     - `Opción 3: Mover en Tablero` -> Actualiza las coordenadas `posicionX` y `posicionY` modificando la matriz 2D.
5. **Turno del Ente Enemigo:** Procesar las acciones automáticas de la inteligencia artificial enemiga.
6. **Limpieza de Bajas:** Invocar `removeIf()` para limpiar los cadáveres de las listas dinámicas al cierre del turno.

4.2 Misiones y Condicionales de Victoria/Derrota

```
================================================================================
                        REQUISITOS DEL RETO INTEGRADOR
================================================================================
[!] Condición de Victoria:
    - Todos los enemigos del escuadrón hostil han sido eliminados (enemigos.isEmpty()
== true).
    - El héroe principal alcanza la posición del cofre [C] en la matriz 2D.

[!] Condición de Derrota:
    - La suma total de la vida de los integrantes del escuadrón héroe es <= 0.
    - La lista heroes.isEmpty() retorna true.

[!] Reglas de Integración Técnica:
    - Ninguna referencia debe perderse por NullPointerException al acceder a celdas
vacías.
    - Todo daño debe reflejarse en la consola y actualizar la matriz del mapa en
tiempo real.
================================================================================
```

Finalizada la implementación de la lógica del juego, procedemos a ejecutar la aplicación en el IDE para diagnosticar y depurar fallos en tiempo de ejecución.

5. Sección 5: Puntos de Depuración e Inspección en IntelliJ IDEA

5.1 Matriz de Excepciones y Errores Comunes

Excepción / Error	Causa Raíz en el Código	Estrategia de Corrección en IntelliJ IDEA
`**IndexOutOfBoundsException**`	Intentar acceder a un índice inexistente en el `ArrayList` o superar los límites de la matriz `tablero[5][5]` al mover un personaje a una posición fuera de rango (ej. índice `5`).	Colocar un *Breakpoint* condicional previo al acceso. Verificar en la pestaña **Variables** el valor exacto del índice y de `lista.size()` o `matriz.length`. Modificar los límites en los condicionales de movimiento.
`**ClassCastException**` / **Errores de Tipado**	Uso inadecuado de tipos crudos (*raw types*) en colecciones o intentar castear explícitamente un objeto `Personaje` a `Mago` cuando la instancia real es un `Guerrero`.	Inspeccionar el objeto en el panel de depuración. Asegurarse de declarar e instanciar siempre usando genéricos explícitos (`Inventario<Pocion>`) para dejar que el compilador capture el error antes del *runtime*.
Error de Compilación: Direct Instantiation	Intentar instanciar directamente la clase base abstracta: `Personaje p = new Personaje();`.	Notar el subrayado rojo del IDE. La corrección consiste en instanciar una subclase concreta (`new Guerrero(...)` o `new Mago(...)`), respetando la regla de que las clases abstractas no se pueden instanciar.

5.2 Guía Práctica de Depuración (Debugging)

Siga esta secuencia para aislar y resolver fallos en IntelliJ IDEA durante la ejecución del taller:

1. **Establecer Puntos de Interrupción (*Breakpoints*):**
   - Haga clic en el margen izquierdo (junto al número de línea) en el método `realizarAtaqueEspecial()` y en el inicio del bucle `do-while` de combate. Aparecerá un punto rojo.
2. **Iniciar el Modo Debug:**
   - Ejecute el programa presionando **Shift + F9** (o haciendo clic en el ícono del escarabajo verde en la esquina superior derecha).
3. **Navegación Paso a Paso:**
   - Utilice **F8 (*Step Over*)** para avanzar línea por línea sin entrar en los métodos.
   - Utilice **F7 (*Step Into*)** para ingresar al código fuente interno de un método (por ejemplo, para ver cómo se calcula el daño dentro de `defender()`).
4. **Inspección del Panel de Variables (*Variables View*):**

- Al pausar la ejecución en un *breakpoint*, explore la pestaña **Variables** en la parte inferior del IDE. Despliegue el objeto `escuadron` para inspeccionar la estructura interna de sus elementos, sus atributos heredados (`vida` , `nombre`) y sus posiciones relativas en la matriz.

5. **Evaluación de Expresiones en Tiempo Real (*Evaluate Expression*):**
   - Presione **Alt + F8** para abrir la ventana de evaluación.
   - Escriba expresiones dinámicas como `escuadron.get(0).estaVivo()` o `tablero[posicionX][posicionY]` para probar hipótesis de fallos sin necesidad de reiniciar la simulación.

Con esta arquitectura definida, las colecciones parametrizadas y las herramientas de depuración a su disposición, está listo para ejecutar, depurar y validar de forma autónoma su proyecto final integrador en IntelliJ IDEA.